

Operating System Concepts

Chapter 3

PROCESSES

Shaon Khan

Based on: Silberschatz, Galvin and Gagne — 9th Edition

Table of Contents

1. Introduction to Processes
2. What is a Process?
3. Process in Memory
4. Process States
5. Process Control Block (PCB)
6. Context Switch
7. Process Scheduling
8. Types of Schedulers
9. Process Types: I/O-Bound vs CPU-Bound
10. Operations on Processes
11. Interprocess Communication (IPC)
12. Chapter Summary

1. Introduction to Processes

1.1 What Will You Learn in This Chapter?

This chapter introduces one of the most fundamental concepts in operating systems: the Process. Every time you open an application on your computer, you are creating a process. Understanding how processes work, how they are managed, and how they communicate with each other is the foundation of understanding any operating system.

By the end of this chapter, you will clearly understand:

- What a process is and how it differs from a program
- How a process is stored in memory
- The different states a process goes through during its lifetime
- How the operating system keeps track of every process
- How the CPU switches between multiple processes
- How processes are scheduled and managed
- How processes are created and terminated
- How processes communicate with each other

1.2 Why Processes Matter

Imagine you are using your computer right now. You have a browser open, music playing, and a document editor running — all at the same time. How does the computer manage all of these activities simultaneously? The answer is: through processes.

The operating system is responsible for creating, managing, and ending processes. Without proper process management, your computer would only be able to do one thing at a time and would freeze or crash frequently.

Real-Life Example

Think of your computer's CPU as a single chef in a kitchen. The chef can only physically cook one dish at a time.

But there are 10 customers waiting. The chef quickly switches between dishes — stirring one pot, then chopping for another, then checking a third — so fast that all customers feel they are being served at the same time.

In a computer, the CPU switches between processes so fast (millions of times per second) that users believe all programs are running simultaneously.

2. What is a Process?

2.1 Program vs. Process

Before we define a process, we must understand the difference between a program and a process. These two words are often confused by beginners.

Term	Explanation
Program	A program is a passive set of instructions stored on disk (e.g., a .exe or .py file). It is not doing anything by itself — it is just sitting there.
Process	A process is an active instance of a program currently being executed by the CPU. When you double-click a program icon, the OS loads it into memory and creates a process.
Simple Analogy	A program is like a recipe written in a book. A process is the actual act of cooking using that recipe. The recipe (program) does nothing on its own; you need a cook (CPU) actively following it to produce food (output).

Key Term: Process

A **process is a program in execution**. It is an active entity that includes the program's code, its current activity (what instruction is being run), and all the data it needs.

2.2 Key Components of a Process

When a program is loaded into memory and started, it becomes a process. A process contains several important components:

1. **Program Counter:** Program Counter (PC)

The program counter is a special register in the CPU that holds the memory address of the next instruction to be executed. It tells the CPU: 'Run this instruction next.'

2. **Stack:** Stack

The stack is a region of memory used for temporary data. Every time a function is called, information is pushed onto the stack (function parameters, return addresses, local variables). When the function ends, that information is popped off.

3. **Data Section:** Data Section

This section holds global variables — variables that are available throughout the entire program (not just inside one function).

4. **Heap:** Heap

The heap is used for dynamic memory allocation. When a program requests memory at runtime (e.g., using `malloc()` in C), that memory comes from the heap.

5. **Text Section (Code Section):** Text Section

This section contains the actual compiled machine instructions of the program — the code that the CPU executes.

2.3 Batch Systems vs. Time-Shared Systems

Historically, operating systems ran programs in two very different ways. Understanding both helps you appreciate how modern systems work.

Feature	Batch System	Time-Shared System
User interaction	None during execution	Continuous / real-time
Programs called	Jobs	Tasks / User programs
Main goal	Maximize throughput (do more jobs)	Maximize responsiveness (serve users quickly)
Users at once	Typically one at a time	Many users simultaneously
Response time	Hours or more / heavily delayed	Seconds / near-immediate
Real-life example	Old mainframe overnight payroll processing	Modern desktop OS, web servers

In modern operating systems (like Windows, Linux, and macOS), time-sharing is the standard. The OS rapidly switches the CPU between many processes, giving every user and every program the feel of having the CPU to themselves.

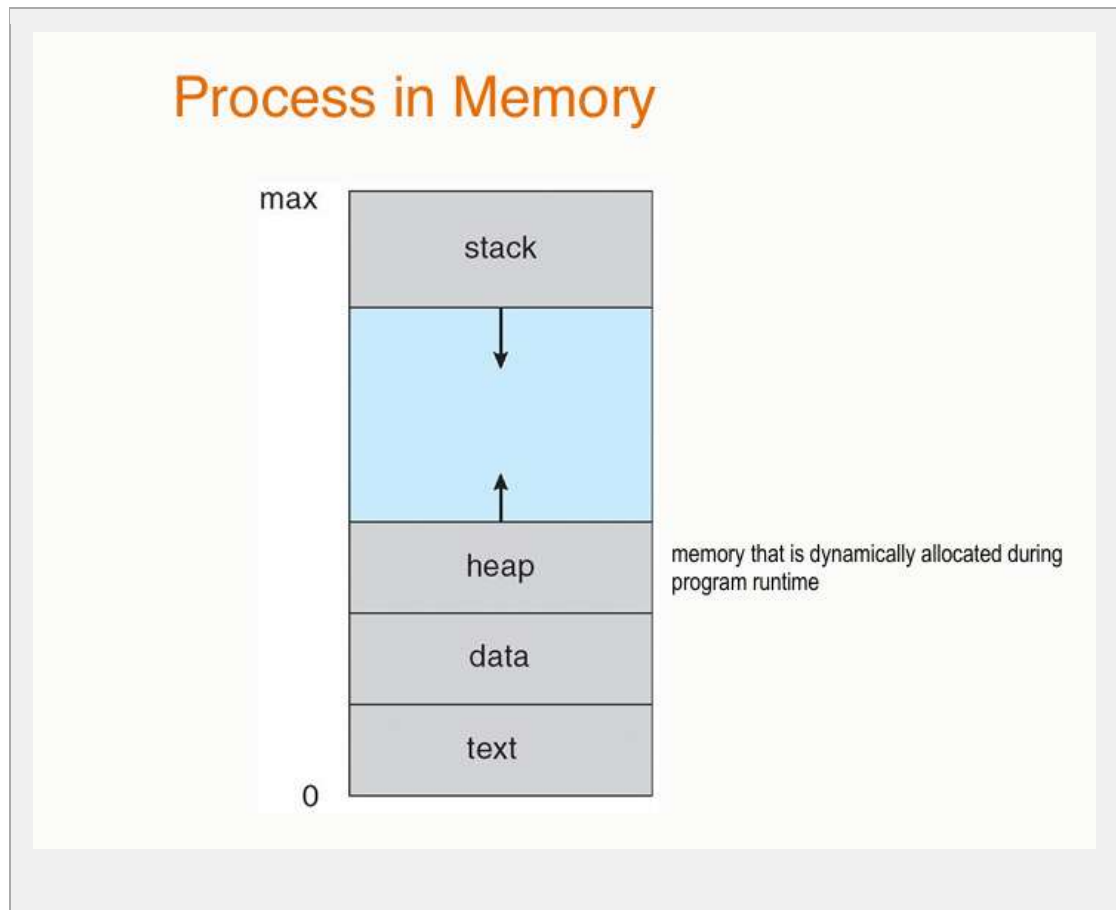
Important Note

Terminology note: The textbook uses the terms 'job' and 'process' almost interchangeably. In batch systems, a running program is called a 'job'; in time-shared systems, it is called a 'task' or 'process'. For this chapter, we will use the term 'process' consistently.

3. Process in Memory

3.1 How a Process is Laid Out in Memory

When a process is running, the operating system allocates a dedicated block of memory for it. This block is **divided into four major sections**, each serving a different purpose. Think of it like a well-organized office with different departments.



The memory layout of a process from the bottom (address 0) to the top (maximum address) is:

Memory Section	Description & Purpose
Text (Code) Section	Located at the lowest memory addresses. Contains the compiled machine instructions — the actual code of the program. This area is typically read-only so no part of the program can accidentally overwrite its own instructions.
Data Section	Sits just above the text section. Stores global and static variables that are initialized before the program starts. These variables exist for the entire lifetime of the process.

Heap	Located above the data section and grows upward toward higher memory addresses. Used for dynamic memory — memory that is requested at runtime (e.g., when you create a large array while the program is running). The programmer controls when heap memory is allocated and freed.
Stack	Located at the highest addresses and grows downward toward the heap. Used for function calls, local variables, and return addresses . Every time a function is called, a 'stack frame' is pushed; when the function returns, the frame is popped. The stack and heap grow toward each other — if they meet, you get a 'stack overflow' error.

Memory Analogy

Imagine a tall building:

- Ground floor (Text): The blueprint — the instructions for how the building works.
- 2nd floor (Data): Permanent offices — always there, used by everyone.
- Middle floors (Heap): Flexible conference rooms — booked as needed and released when done.
- Top floors (Stack): Temporary workspaces — used for active tasks, cleared when the task finishes.

4. Process States

4.1 The Life of a Process

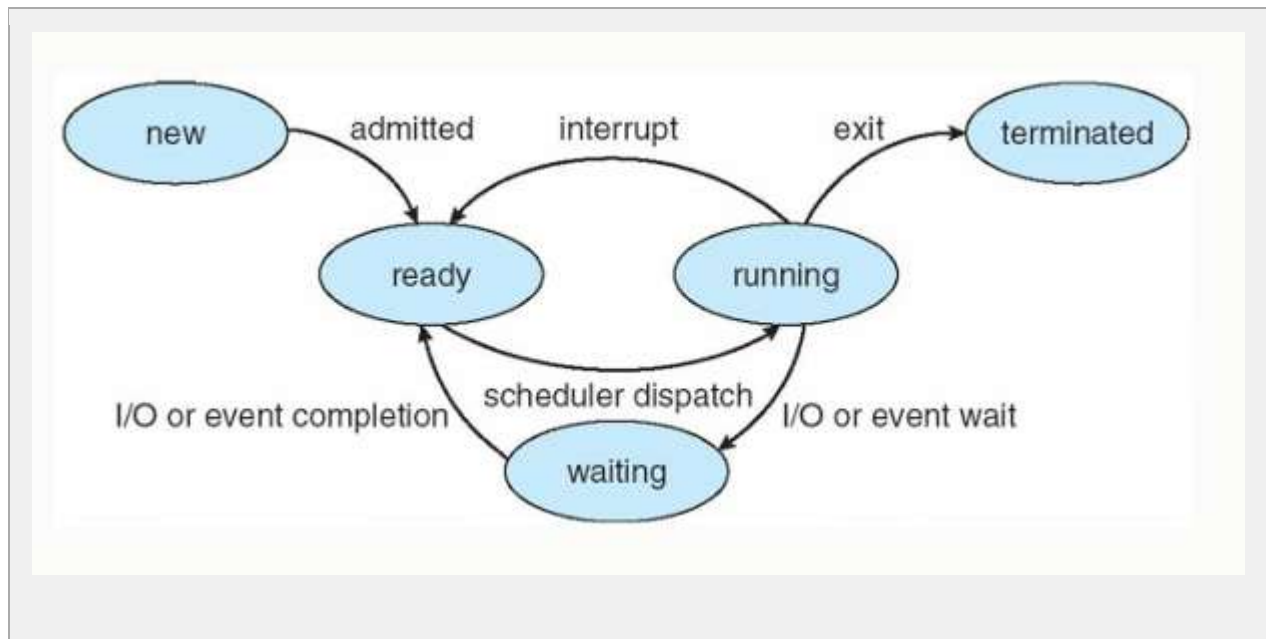
A process does not simply start running and then stop. It goes through several distinct states during its lifetime. These states tell the operating system exactly what a process is doing at any given moment.

Think of a person waiting in a doctor's office:

- They arrive and check in (New)
- They sit in the waiting room (Ready)
- They are called in and the doctor sees them (Running)
- The doctor orders a test and they wait for results (Waiting)
- They leave after treatment (Terminated)

A process follows a very similar journey.

4.2 The Five Process States



State	What is Happening
New	The process is being created. The OS is setting up its memory, PCB, and initial resources. It has not yet been admitted to the ready queue.

Ready	The process is fully prepared to run. It is loaded in memory and waiting for the CPU to become available. Multiple processes can be in the Ready state at the same time.
Running	The process is actively using the CPU. Instructions are being executed right now. On a single-core system, only ONE process can be in the Running state at a time.
Waiting (Blocked)	The process cannot continue until some event occurs — usually an I/O operation (e.g., reading from disk, waiting for keyboard input). The CPU is freed up for other processes while this process waits.
Terminated	The process has finished execution. The OS reclaims all resources (memory, open files, etc.) that the process was using.

4.3 How Processes Move Between States

The transitions between states are triggered by specific events:

1. New → Ready: The OS admits the process and places it in the ready queue.
2. Ready → Running: The CPU scheduler selects this process and dispatches it to the CPU.
3. Running → Waiting: The process requests an I/O operation or waits for an event (e.g., reads a file). It voluntarily gives up the CPU.
4. Waiting → Ready: The I/O operation completes, or the awaited event occurs. The process goes back to the ready queue.
5. Running → Ready: The process is interrupted (preempted) by the OS — perhaps because its time slice expired or a higher-priority process needs the CPU.
6. Running → Terminated: The process finishes execution or is killed by the OS.

Real-Life Scenario

You open a word processor (New → Ready → Running).

You click 'Open File' — the app waits for the disk to load the file (Running → Waiting).

The file loads (Waiting → Ready).

The OS gives the CPU back to the word processor (Ready → Running).

You close the app (Running → Terminated).

5. Process Control Block (PCB)

5.1 What is a PCB?

The operating system must keep detailed records about every process it is managing. It does this using a data structure called the Process Control Block (PCB), sometimes also called the **Task Control Block**.

Process Control Block (PCB)

A PCB is a data structure maintained by the operating system for each process. It contains all the information the OS needs to manage, control, and resume that process at any time.

Think of the PCB as the 'employee file' the OS keeps for each process. Just like an HR department keeps records about every employee (name, department, salary, performance), the OS keeps records about every process in its PCB.

5.2 What Information is Stored in a PCB?

The PCB contains seven major categories of information:

PCB Field	What it Contains & Why it Matters
Process State	The current state of the process: new, ready, running, waiting, or terminated. The OS uses this to know if the process needs the CPU, is waiting for I/O, etc.
Program Counter (PC)	The address of the next instruction that this process needs to execute. When the OS pauses a process and later resumes it, it reads this value to know exactly where to continue.
CPU Registers	The values of all CPU registers (accumulators, index registers, stack pointers, etc.) at the time the process was last paused. These must be saved and restored during a context switch so computation can continue correctly.
CPU Scheduling Information	Details used by the scheduler: process priority, pointers to scheduling queues, and other parameters. A high-priority process (like a real-time audio player) will be scheduled before a low-priority one (like a background backup).
Memory Management Information	Information about the memory allocated to this process: base and limit registers, page tables, or segment tables. This prevents one process from accessing another process's memory.

Accounting Information	Statistics about the process: how much CPU time it has consumed, wall-clock time elapsed, job/process number, time limits. Used for billing in multi-user systems and for performance monitoring.
I/O Status Information	A list of I/O devices currently allocated to the process (e.g., a printer), and a list of open files. This ensures the OS knows what resources to free when the process terminates.

 Important Note

Every process on your computer has its own PCB. On a modern system, there may be hundreds of PCBs maintained simultaneously. You can see many of them by opening the Task Manager (Windows) or Activity Monitor (macOS) and viewing the list of running processes.

6. Context Switch

6.1 What is a Context Switch?

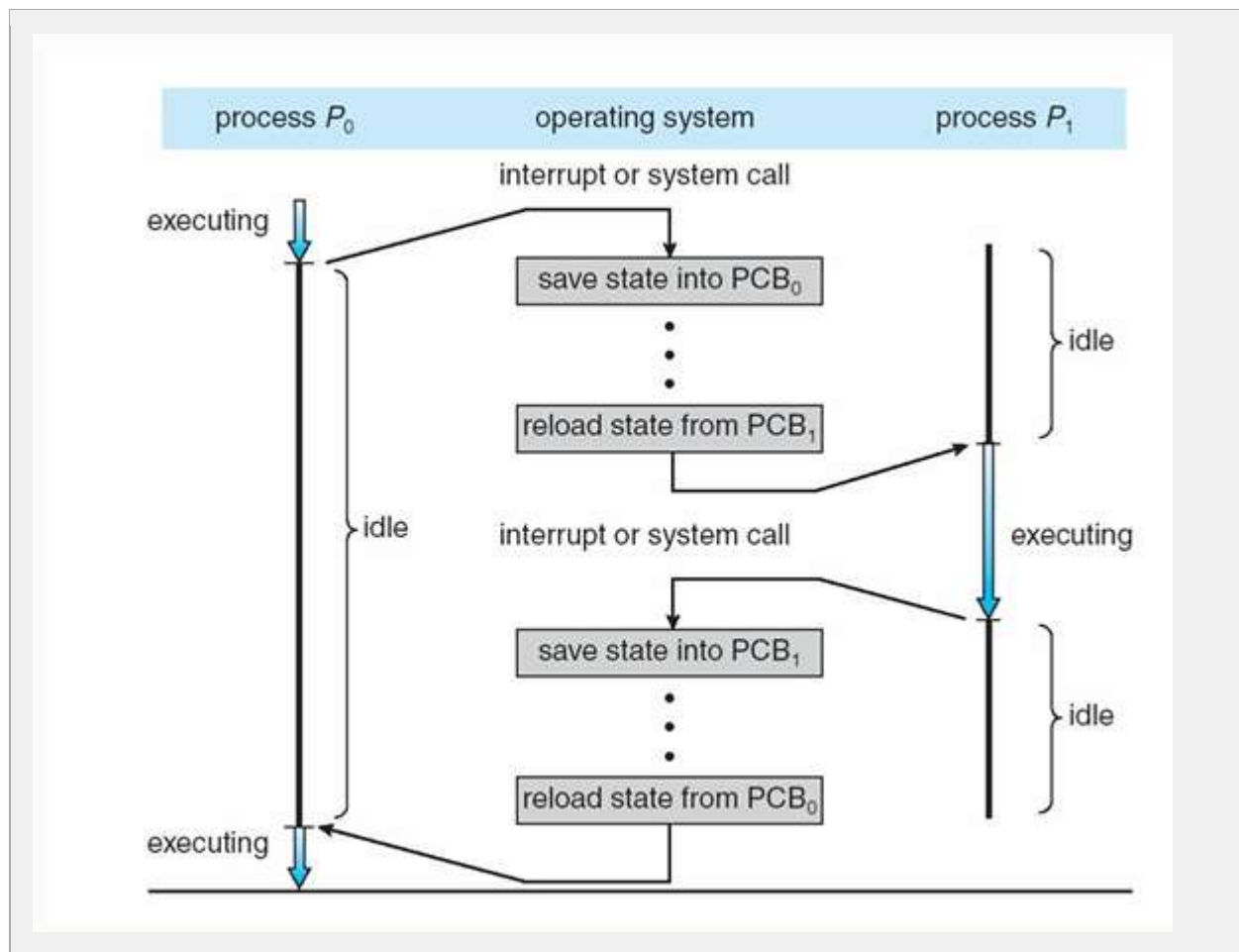
Since most computers have only one CPU core (or a limited number), the CPU must be shared among many processes. To switch from running one process to running another, the operating system performs a context switch.

Key Term: Context Switch

A context switch is the process of saving the current state (context) of a running process into its PCB, and then loading the saved state of another process from its PCB so that process can run.

6.2 Step-by-Step: How a Context Switch Works

Let us walk through what happens when the CPU switches from Process P0 to Process P1:



1. Process P0 is currently running on the CPU.
2. An interrupt or system call occurs (e.g., P0 requests I/O or a timer expires).
3. The OS saves P0's complete context — program counter, all CPU registers, and state — into PCB0.
4. The OS selects the next process to run (P1) from the ready queue.
5. The OS loads P1's saved context from PCB1 into the CPU registers.
6. The CPU resumes executing P1 from exactly where it last stopped.
7. When P1 is later paused, its context is saved into PCB1, and P0's context is reloaded from PCB0, so P0 can resume.

6.3 The Cost of Context Switching

Context switching is not free — it takes time. During a context switch, the CPU is doing administrative work (saving and loading PCBs) instead of running application code. This is called overhead.

Factor	Impact on Context Switch Time
OS complexity	More complex OS means more information to save/restore → slower switches
PCB size	Larger PCBs take longer to save and restore
Hardware support	Some CPUs have special instructions to speed up register saves → faster switches
Number of registers	More registers = more data to save/restore → slightly slower

Analogy: The Bookmark

A context switch is like using a bookmark in a book.

When you stop reading one book (process), you place a bookmark (save context to PCB) exactly where you stopped.

You then pick up another book and open it at your saved bookmark (load context from PCB).

Later, you can return to the first book and continue from exactly where you left off.

The bookmark itself takes a tiny amount of time to place and find — that is the overhead.

7. Process Scheduling

7.1 Why Do We Need Scheduling?

At any given moment, many processes may be ready to use the CPU. But there is only one CPU (or a limited number of cores). The OS must decide: which process gets to run next, and for how long?

This decision-making is called process scheduling. The main goals of scheduling are:

- **Maximize CPU utilization** — keep the CPU busy as much as possible
- **Maximize throughput** — complete as many processes as possible per unit time
- **Minimize waiting time** — reduce how long processes sit in the ready queue
- **Ensure fairness** — every process gets a reasonable share of the CPU

7.2 Scheduling Queues

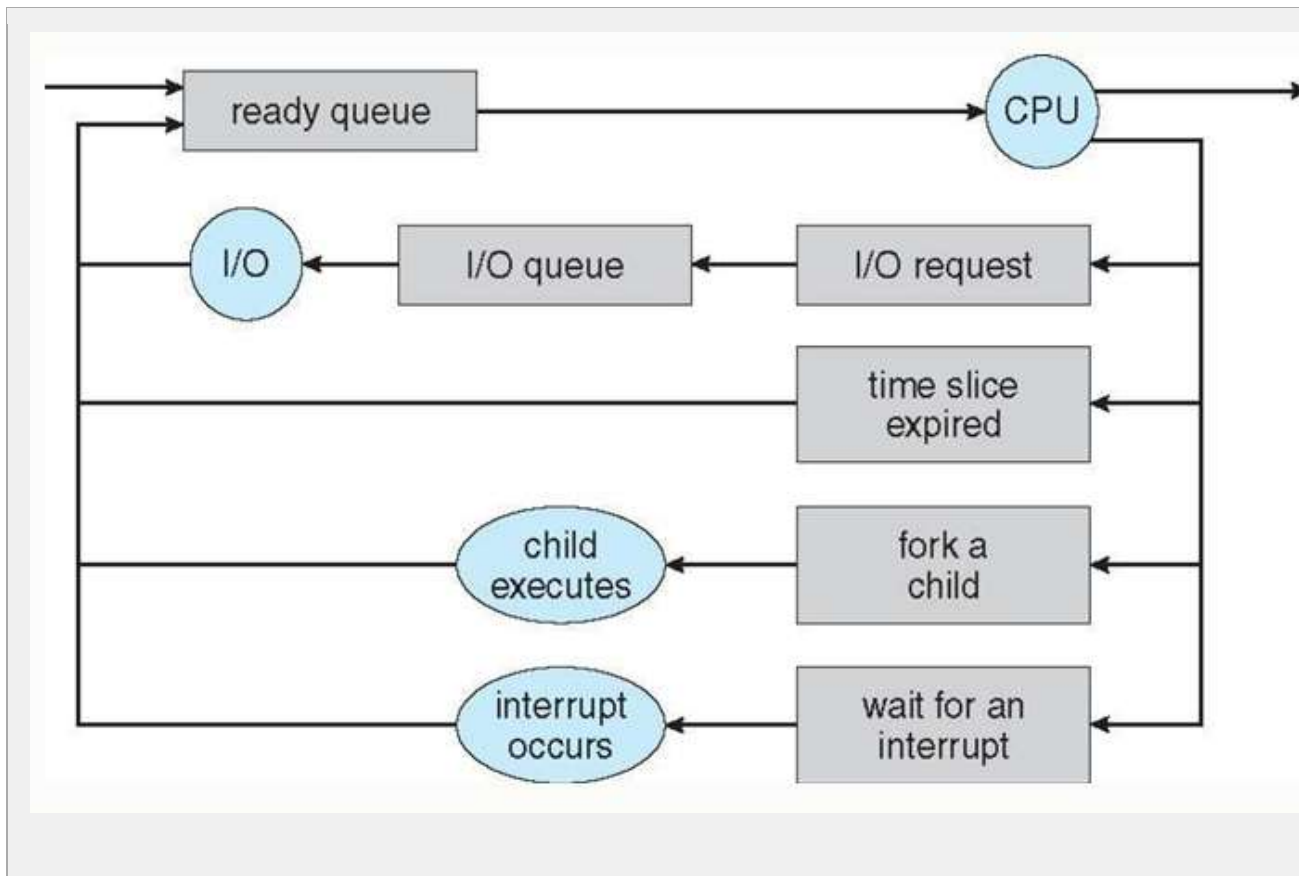
The OS maintains several queues to organize processes at different stages:

Queue Name	What It Contains
Job Queue	All processes in the entire system — regardless of their current state. This is the master list.
Ready Queue	All processes that are in main memory, fully ready, and waiting to be assigned to the CPU. This is typically a linked list of PCBs.
Device Queues	Processes waiting for a specific I/O device (one queue per device). For example, all processes waiting to read from the disk form the disk queue.

Processes constantly migrate between these queues as they change state. A process enters the ready queue when it is created or finishes waiting, and leaves it when it gets the CPU.

7.3 What Happens When a Process is Running?

Once the OS assigns the CPU to a process, several things can happen next:



1. **I/O Request:** The process needs to read or write data (e.g., open a file). It is moved from running to an I/O device queue.
2. **Create a Child Process:** The process creates a new subprocess and waits for it to finish. The parent is moved to a wait state.
3. **Time Slice Expiry:** The process has used its allotted time on the CPU. The OS preempts it and places it back in the ready queue so another process can run.
4. **Interrupt:** An external event (hardware interrupt) forces the process off the CPU. It returns to the ready queue.

8. Types of Schedulers

8.1 Overview of the Three Schedulers

Not all scheduling decisions are the same. Some decisions happen very frequently; others happen rarely. Operating systems use three types of schedulers to handle different levels of scheduling:

Scheduler Type	What It Does	How Often
Long-Term (Job) Scheduler	Decides which processes from the job pool (on disk) should be brought into main memory (ready queue). Controls the degree of multiprogramming.	Very infrequently — seconds or minutes apart
Short-Term (CPU) Scheduler	Decides which process in the ready queue should be given the CPU next. This is the most critical scheduler.	Very frequently — every few milliseconds
Medium-Term Scheduler	Temporarily removes processes from memory (swaps them to disk) to reduce the load, then brings them back later. Performs 'swapping'.	Occasionally, when memory is overloaded

8.2 Long-Term Scheduler — The Gatekeeper

The long-term scheduler (also called the job scheduler) controls how many processes are in memory at any time. This number is called the degree of multiprogramming.

Scenario	Effect
Degree too LOW	CPU sits idle. Resources are wasted. The system is underloaded.
Degree too HIGH	Memory becomes overloaded. Processes compete for scarce memory. Performance drops significantly.
Degree BALANCED	CPU stays busy while memory is not overloaded. The system runs at optimal efficiency.

The long-term scheduler carefully selects a mix of CPU-bound and I/O-bound processes (explained in the next section) to keep both the CPU and I/O devices busy.

8.3 Short-Term Scheduler — The Fast Decision Maker

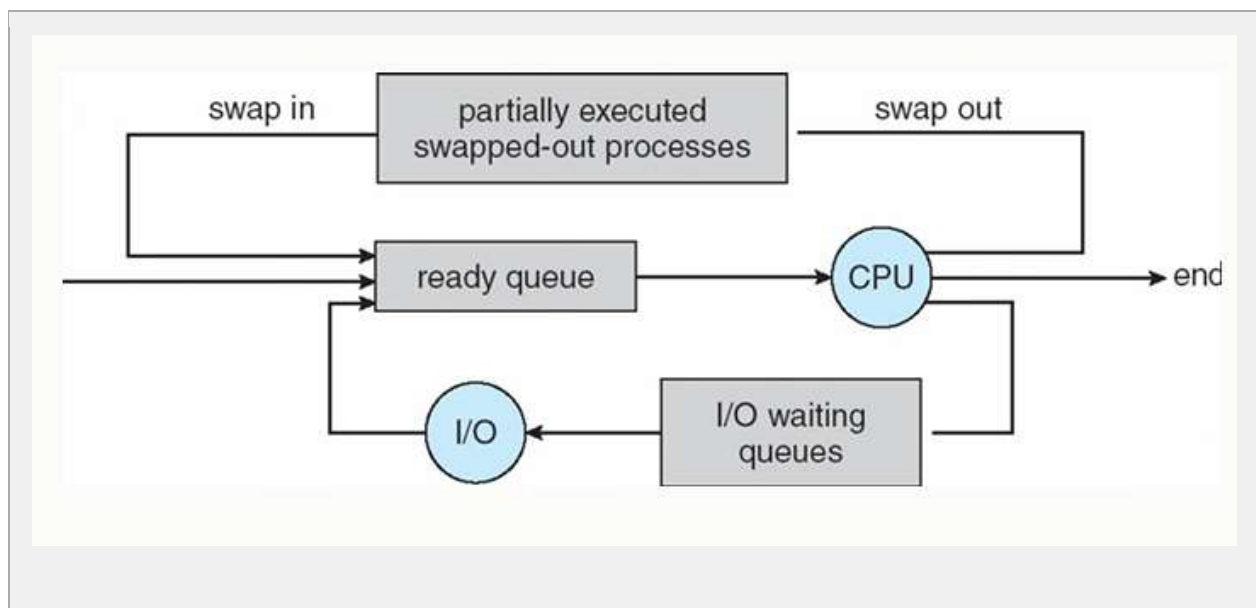
The short-term scheduler (CPU scheduler) must make decisions extremely quickly because it runs every few milliseconds. This scheduler directly determines which process runs on the CPU next.

Key characteristics:

- Must be very fast — any delay directly reduces CPU efficiency
- Invoked on every timer interrupt, I/O completion, and system call
- Is the most important scheduler in modern systems
- In many simple systems, it is the ONLY scheduler

8.4 Medium-Term Scheduler — The Memory Manager

Sometimes the system becomes overloaded with too many processes in memory. The medium-term scheduler solves this problem through a technique called swapping.



Swapping

Swapping is the process of moving a process from main memory (RAM) to secondary storage (disk) — called swap out — and later moving it back to memory — called swap in — so it can continue running.

Swapping is useful when:

- Memory is full and a new high-priority process needs to be loaded
- A process has been inactive for a long time and is unlikely to run soon
- The system needs to reduce the degree of multiprogramming temporarily

9. Process Types: I/O-Bound vs. CPU-Bound

9.1 Why This Classification Matters

Not all processes use the CPU the same way. Some processes spend most of their time waiting for input/output operations; others spend most of their time doing computation. This difference is important for the scheduler.

CPU Burst

A CPU burst is a period of time during which a process is actively using the CPU to execute instructions — before it needs to pause and wait for something (like I/O). Every process alternates between CPU bursts and I/O wait periods.

9.2 Two Types of Processes

Type	Characteristics
I/O-Bound Process	Spends more time doing I/O operations than computation. Has many SHORT CPU bursts with frequent I/O waits. Examples: word processor, web browser, file copy, database query.
CPU-Bound Process	Spends more time computing than waiting for I/O. Has few but LONG CPU bursts. Examples: video encoding, scientific simulation, machine learning training, encryption.

9.3 Why the Mix Matters for the Scheduler

The long-term scheduler should maintain a good mix of both types:

- Too many CPU-bound processes: CPU is hogged; I/O devices sit idle; I/O-bound processes starve.
- Too many I/O-bound processes: CPU sits idle most of the time; throughput drops.
- A balanced mix: CPU and I/O devices both stay busy; maximum system efficiency.

Example Scenario

Process A is CPU-bound: it runs [====long burst====] then waits briefly, then [====long burst====] again.

Process B is I/O-bound: it runs [=short=] then waits for disk, [=short=] waits for keyboard, [=short=] waits for network.

If only Process A runs: the CPU is busy but disk, keyboard, and network sit idle.

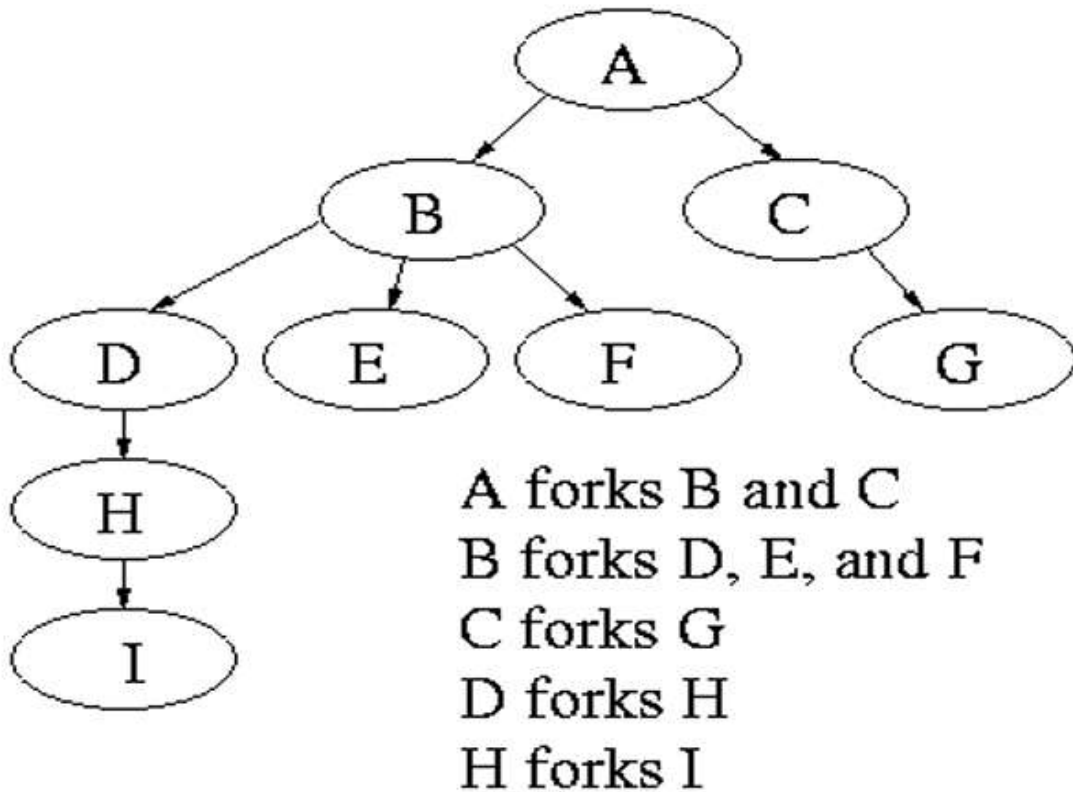
If both run together: while Process A uses the CPU, Process B's I/O operations complete in the background. When A pauses for I/O, B can briefly use the CPU. The whole system is utilized efficiently.

10. Operations on Processes

10.1 Process Creation

Operating systems allow processes to create new processes. The creating process is called the parent process, and the new process it creates is called the child process. This parent-child relationship forms a tree of processes.

A Tree of Processes in Linux



Process Identification (PID)

Every process in the system is uniquely identified by a Process Identifier (PID) — a non-negative integer assigned by the OS. When you look at Task Manager or run the 'ps' command in Linux, you see PIDs next to each process name.

Resource Sharing Options

When a parent creates a child process, there are three possibilities for how they share resources:

6. Parent and child share ALL resources (full sharing).
7. Child shares a SUBSET of the parent's resources.
8. Parent and child share NO resources (completely independent).

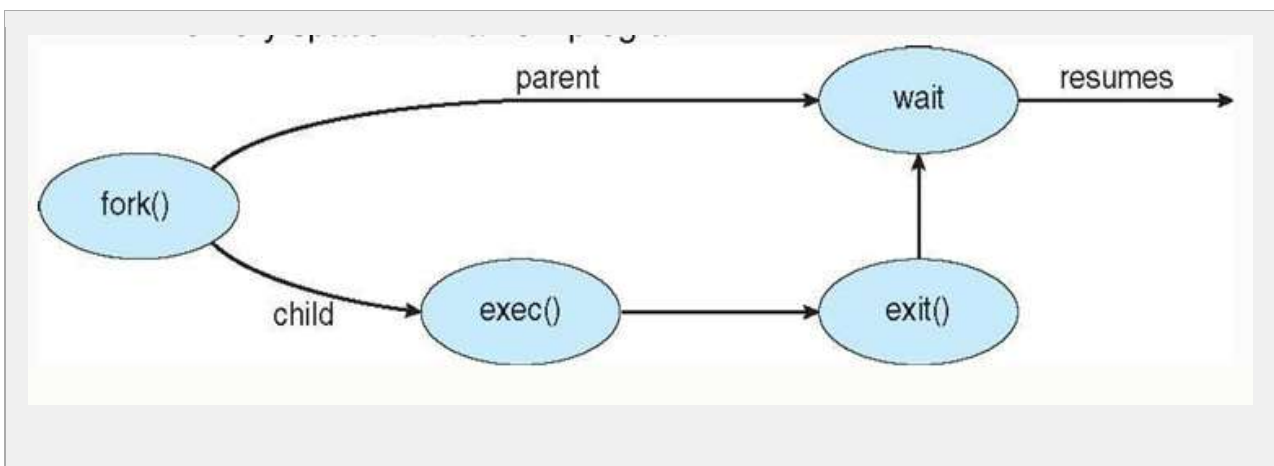
Execution Options

After creation, the parent and child can interact in two ways:

5. Concurrent Execution: Parent and child execute at the same time (independently).
6. Parent Waits: Parent pauses and waits until the child process terminates before continuing.

10.2 Process Creation in UNIX/Linux

In UNIX-based systems, new processes are created using two key system calls:



System Call	What It Does
fork()	Creates an exact duplicate of the calling process. The child is a copy of the parent — same code, same data, same open files. After fork(), both parent and child continue running from the same point in the code.
exec()	Replaces the current process's memory space with a new program. After fork(), the child typically calls exec() to load and run a different program entirely. The child's memory is overwritten with the new program's code and data.
wait()	The parent calls wait() to pause itself until the child finishes. When the child terminates, the parent is notified and can collect the child's exit status.

<code>exit()</code>	A process calls <code>exit()</code> when it wants to terminate. This triggers cleanup and notifies the parent.
---------------------	--

Real-Life Example

When you type `'ls -la'` in a Linux terminal:

1. The terminal (shell) calls `fork()` → creates a child process that is a copy of the shell.
2. The child calls `exec('ls')` → replaces itself with the `'ls'` program.
3. The `'ls'` program runs and prints the directory listing.
4. `'ls'` calls `exit()` → terminates.
5. The shell's `wait()` returns → the shell prompts you for the next command.

This fork-exec pattern is used for virtually every command you run in a terminal.

10.3 Process Termination

A process can end in several ways:

Normal Termination

The process reaches its last statement and calls `exit()`. Before the process disappears, it can send its output data to its parent process (via `wait()`). The OS then de-allocates all resources held by the process: memory, open files, I/O devices.

Parent-Initiated Termination (`abort()`)

A parent process can forcefully terminate a child process using the `abort()` system call. This happens when:

- The child has exceeded its allocated resource limits (using too much memory or CPU time)
- The task assigned to the child is no longer needed
- The parent itself is exiting

Cascading Termination

In some operating systems, if a parent process terminates, all its children are automatically terminated too (since an orphan process would have no parent to manage it). This chain of terminations is called cascading termination.

Important Note

Not all operating systems implement cascading termination. Linux, for example, allows 'orphan' processes to continue running by re-parenting them to the `init` process (PID 1), which then acts as their new parent.

10.4 Real-World Example: Google Chrome's Multi-Process Architecture

Google Chrome is an excellent real-world example of a multi-process application. Early web browsers ran as a single process, which meant that if one website caused a crash, the entire browser would crash.

Chrome solved this problem by using three categories of processes:

Chrome Process Type	Responsibility
Browser Process (1 instance)	Manages the overall user interface, handles disk I/O, and manages network connections. There is exactly one Browser process.
Renderer Process (one per tab)	Renders web pages — handles HTML, CSS, JavaScript, and DOM. Each website you open gets its own Renderer process, isolated from others. If one tab crashes, only that tab is affected; the browser and other tabs continue normally.
Plug-in Process (one per plug-in type)	Each browser plug-in (e.g., Flash, PDF viewer) runs in its own process. This prevents a buggy plug-in from crashing the entire browser.

Why Multi-Process Matters

Isolation: A crash in one renderer process does not affect others.

Security: Each renderer runs in a restricted sandbox, limiting what it can access.

Stability: Users can continue browsing other tabs even if one crashes.

This is why Chrome uses more memory than some other browsers — each tab is its own process with its own copy of memory.

11. Interprocess Communication (IPC)

11.1 Independent vs. Cooperating Processes

Processes running on a system can be classified as either independent or cooperating:

Type	Description
Independent Process	A process that does not share data or communicate with any other process. Its execution does not affect, and is not affected by, any other process. Example: a standalone calculator app.
Cooperating Process	A process that can affect or be affected by other processes. Cooperating processes share data or must coordinate their actions. Example: a web server that forks a child to handle each incoming request.

11.2 Why Allow Process Cooperation?

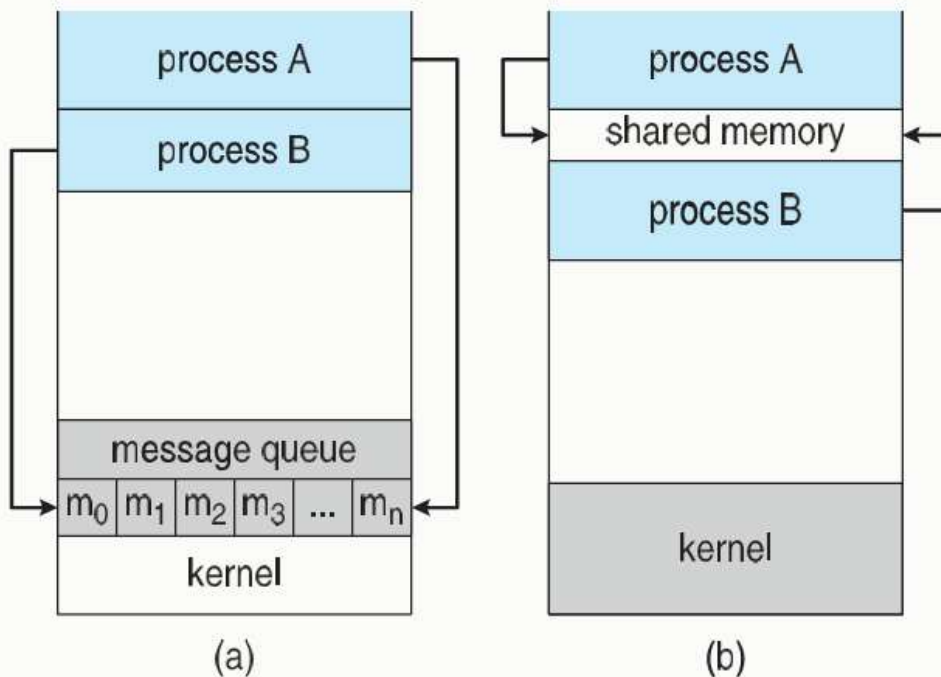
There are four major reasons why it is useful to allow processes to cooperate:

9. Information Sharing: Multiple users or processes may need access to the same data (e.g., a shared file or database). Cooperation allows concurrent access to shared resources.
10. Computation Speed-Up: A large task can be broken into subtasks that run in parallel on multiple CPU cores. For example, rendering different parts of an image simultaneously. This is why video editors and compilers can use all your CPU cores.
11. Modularity: A complex system can be divided into smaller, specialized processes. Each process does one job well and communicates with others as needed. This makes systems easier to design, maintain, and update.
7. Convenience: A single user may want to run multiple tasks simultaneously — downloading a file, playing music, and writing a document — all at the same time.

11.3 The Two Models of IPC

Cooperating processes need a mechanism to communicate. There are two fundamental models of Interprocess Communication:

Communications Models



Model 1: Shared Memory

In the shared memory model, a region of memory is created that both processes can directly read and write. The OS sets up the shared region; after that, communication happens at memory speed (very fast) without the OS being involved in every exchange.

- How it works: Process A writes data to the shared memory region. Process B reads it.
- Speed: Very fast — no kernel involvement after initial setup.
- Challenge: Requires synchronization. If both processes try to write at the same time, data can get corrupted. This is called a race condition and requires careful handling using locks or semaphores (covered in later chapters).
- Best for: Transferring large amounts of data between processes on the same machine.
- Real-life example: A video encoding program splits a video into segments, shares the frame buffer in memory with multiple worker processes, and each worker processes its own segment simultaneously.

Model 2: Message Passing

In the message passing model, processes communicate by sending and receiving messages through the operating system kernel. The kernel acts as a post office — it receives messages from one process and delivers them to another.

- How it works: Process A calls `send(message)`. The kernel places the message in a queue. Process B calls `receive()` and gets the message.
- Speed: Slower than shared memory — every message requires a system call (kernel involvement).
- Advantage: No shared memory region to manage. No risk of race conditions. Easier to use correctly.
- Best for: Passing small amounts of data, especially between processes on different computers (distributed systems).
- Real-life example: A web server receives an HTTP request (message from client process), processes it, and sends back an HTTP response (message to client). This communication uses message passing (sockets).

11.4 Comparing the Two IPC Models

Feature	Shared Memory	Message Passing
Speed	Very fast (memory speed)	Slower (system calls required)
Data volume	Large data transfers	Small data / short messages
Synchronization needed?	Yes — must use locks/semaphores	No — handled by OS
OS involvement after setup	Minimal	For every message
Ease of use	More complex (race conditions)	Simpler to use correctly
Typical use case	Same machine, large data sharing	Any distance, small messages
Real-world example	Shared frame buffer in video editing	Client-server web communication

12. Chapter Summary

Key Concepts at a Glance

Concept	One-Line Summary
Process	A program in active execution; contains code, data, stack, heap, and a program counter.
Process vs. Program	A program is static instructions on disk; a process is a running instance in memory.
Memory Layout	Text (code) → Data (globals) → Heap (dynamic, grows up) → Stack (function calls, grows down).
Process States	New → Ready → Running → Waiting → Terminated; the OS controls all transitions.
PCB	The OS's complete data file for a process: state, PC, registers, memory info, I/O info, accounting.
Context Switch	Save current process's PCB, load next process's PCB; pure overhead — no useful work done.
Process Scheduling	The OS chooses which ready process gets the CPU next, using scheduling queues.
Long-Term Scheduler	Controls how many processes are in memory; runs infrequently; manages degree of multiprogramming.
Short-Term Scheduler	Picks which ready process runs next; runs every millisecond; must be very fast.
Medium-Term Scheduler	Swaps processes in/out of memory when system is overloaded.
I/O-Bound Process	Many short CPU bursts; spends most time waiting for I/O.
CPU-Bound Process	Few long CPU bursts; spends most time computing.
fork() / exec()	fork() duplicates a process; exec() replaces it with a new program. Standard UNIX process creation.
Cascading Termination	When a parent dies, all its children may also be terminated automatically.
IPC — Shared Memory	Processes share a memory region directly; fast but needs synchronization.
IPC — Message Passing	Processes exchange messages via the OS kernel; simpler but slower.

Chapter Learning Outcome

After studying this chapter, you should be able to:

- Explain what a process is and how it differs from a program
- Describe the five-section memory layout of a process
- Identify and explain all five process states and their transitions
- Describe the contents of a Process Control Block
- Explain what a context switch is and why it creates overhead
- Describe the three types of schedulers and their roles
- Explain I/O-bound and CPU-bound processes and why the distinction matters
- Describe process creation and termination in UNIX systems using fork/exec
- Compare and contrast the two models of Interprocess Communication

— End of Chapter 3: Processes —

Operating System Concepts, 9th Edition • Silberschatz, Galvin and Gagne